Device Firmware Upgrade via USB

[中文]

Typically, the firmware of ESP32-S3 is flashed via the chip's serial port or USB_SERIAL_JTAG (see Establish Serial Connection with ESP32-S3 for more details). ESP32-S3 also contains a USB OTG peripheral making it possible to connect ESP32-S3 to the host directly via USB Device Firmware Upgrade.

By default, the USB_SERIAL_JTAG module is connected to ESP32-S3's internal USB PHY, while the USB OTG peripheral can be used only if an external USB PHY is connected. Since DFU is provided via the USB OTG peripheral, it cannot be used through the internal PHY in this configuration.

However, you can permanently switch the internal USB PHY to work with USB OTG peripheral instead of USB_SERIAL_JTAG by burning the `USB_PHY_SEL` eFuse. See *ESP32-S3 Technical Reference Manual* [PDF] for more details about USB_SERIAL_JTAG and USB OTG.

For a practical example of USB OTG implementation, refer to the ESP32-S3-USB-OTG development board, which is specifically designed for USB OTG applications.

Device Firmware Upgrade (DFU) is a mechanism for upgrading the firmware of ESP32-S3 directly via the Universal Serial Bus (USB). However, enabling Secure Boot or flash encryption disables the USB-OTG USB stack in the ROM, disallowing updates via the serial emulation or DFU on that port.

- Software of the Getting Started Guide introduces the software requirements of DFU.
- Section Building the DFU Image describes how to build firmware for DFU with ESP-IDF.
- Section Flashing the DFU Image deals with flashing the firmware.

USB Connection

The necessary connections for ESP32-S3's internal USB PHY (transceiver) are shown in the following table:

GPIO	USB
20	D+ (green)
19	D- (white)
GND	GND (black)

GPIO	USB
+5V	+5V (red)

Warning

Some cables are wired up with non-standard colors and some drivers are able to work with swapped D+ and D- connections. Please try to swap the cables connecting to D+ and D- if your device is not detected.

Note

ESP32-S3 chip needs to be in bootloader mode before it can be detected as a DFU device and flash. Please refer to Boot Mode Selection for more information about how to enter bootloader mode.

Building the DFU Image

The command below will create a DFU image named `dfu.bin` that is placed in the project's `build` directory:

```
idf.py dfu
```

Note

Do not forget to set the target chip by `idf.py set-target` before running `idf.py dfu`. Otherwise, you might create an image for a different chip or receive an error message like `unknown target 'dfu'`.

Flashing the DFU Image

The command below will download the DFU image into ESP32-S3:

```
idf.py dfu-flash
```

The command relies on dfu-util. Please see Software for installing `dfu-util`. `dfu-util` needs additional setup for USB Drivers (Windows Only) or setting up an Udev Rule (Linux Only). macOS users should be able to use `dfu-util` without further setup.

If there are more boards with the same chip connected then `idf.py dfu-list` can be used to list the available devices, for example:

```
Found Runtime: [303a:0002] ver=0723, devnum=4, cfg=1, intf=2, path="1-10", alt=0,
name="UNKNOWN", serial="0"
Found Runtime: [303a:0002] ver=0723, devnum=6, cfg=1, intf=2, path="1-2", alt=0,
name="UNKNOWN", serial="0"
```

Consequently, the desired device can be selected for flashing by the `--path` argument. For example, the devices listed above can be flashed individually by the following commands:

```
idf.py dfu-flash --path 1-10
idf.py dfu-flash --path 1-2
```

Note

The vendor and product identificators are set based on the selected chip target by the `idf.py set-target` command and they are not selectable during the `idf.py dfu-flash` call.

See Common Errors and Known Issues and their solutions.

Udev Rule (Linux Only)

Udev is a device manager for the Linux kernel. It allows running `dfu-util` (and `idf.py dfu-flash`) without `sudo` for gaining access to the chip.

Create file `/etc/udev/rules.d/40-dfuse.rules` with the following content:

```
SUBSYSTEMS=="usb", ATTRS{idVendor}=="303a", ATTRS{idProduct}=="00??", GROUP="plugdev",
MODE="0666"
```

Note

Please check the output of the command `groups`. You need to be a member of the *GROUP* specified above. You may use some other existing groups for this purpose (e.g., *uucp* on some systems instead of *plugdev*) or create a new group for this purpose.

Restart your computer so the previous setting could take into affect or run `sudo udevadm trigger` to force manually udev to trigger your new rule.

USB Drivers (Windows Only)

`dfu-util` uses *libusb* to access the device. On Windows, the *WinUSB* driver is the recommended driver which has to be installed for the device to work properly. For more details please see the libusb wiki.

Note

If the feature is not working please proceed with the manual driver assignment. Otherwise, the following section can be skipped.

USB Drivers (Windows Only) - manual driver assignment

Manual driver assignment can be performed with the Zadig tool. Please make sure that the device is in download mode before running the tool and that the ESP32-S3 device is detected before the driver installation.

The Zadig tool might detect several USB interfaces of ESP32-S3. Please install the *WinUSB* driver **only** for the interface where there is no driver installed (probably it is Interface 2) and do not re-install the driver for the other interface.

Warning

The manual installation of the driver in Device Manager of Windows is not recommended because the flashing might not work properly.

Common Errors and Known Issues

- `dfu-util: command not found` might indicate that the tool has not been installed or is not available from the terminal. An easy way of checking the tool is running `dfu-util --version`. Please see Software for installing `dfu-util`.
- The reason for `No DFU capable USB device available` could be that the USB driver was not properly installed on Windows (see USB Drivers (Windows Only)), udev rule was not setup on Linux (see Udev Rule (Linux Only)) or the device is not in bootloader mode.
- Flashing with `dfu-util` on Windows or macOS may fail on the first attempt with error `Lost device after RESET?`. `idf.py dfu-flash` retries once automatically. If flashing still fails, run `idf.py dfu-flash` again manually.

Secure Download Mode

When Secure Download Mode is enabled, DFU is no longer possible. Please see Flash Encryption

guide for more details.

Was this page helpful?



Thank you! We received your feedback.
If you have any comments, fill in Espressif Documentation Feedback Form.